Advanced computerarchitecture
Exercise 2 Report

(B-KUL-JPI30R)

Wout Van Steenbergen

February 27, 2024



Session Date: February 19, 2024

Instructor: Alex Marinšek

1 Introduction

In the previous discussion (Exercise 1 report), the usability of CUDA was explored for its capability to harness the parallel computing powers of NVIDIA GPU's. Building upon that, exercise 2 explores the possibilities of max-detection within an array using three different methods. Initially, a nested for loop executed on the CPU is employed, serving as the baseline for comparison. Subsequently, a GPU kernel utilizing atomic operations is implemented. And finally a GPU kernel designed around the reduction technique, aimed at efficient aggregation with minimized overhead.

2 CPU Method: Nested For-Loop

This method uses a straightforward approach where each element of the array is iterated over in a sequential manner to find the maximum value. The findMaxCPU function takes an array and its size as inputs. It initializes the maximum value to the smallest integer possible (INT_MIN) and iterates through each element of the array. If an element is found to be greater than the current maximum, the maximum is updated to this new value. This process continues until all elements have been checked.

3 GPU Method: Atomic Operations

The second method utilizes the GPU for parallel computation and specifically employs atomic operations to find the maximum value. The findMaxAtomicGPU kernel function is designed to run on multiple GPU threads concurrently. Each thread calculates its own index based on the same expression as in the previous report [int index = blockIdx.x * blockDim.x + threadIdx.x;]. Both the number of threads and blocks are also calculated the same. The thread compares the element at its index with a global maximum value using the atomicMax function. The atomicMax function ensures that updates to the maximum value are done atomically, preventing race conditions between threads.

4 GPU Method: Reduction

The third approach also utilizes the GPU but employs a reduction algorithm to find the maximum value. The reduction algorithm is a parallel algorithm that reduces the input data set to a smaller set of results (in this case, the maximum value). First each thread calculates its unique index in the same way as above. Next, a stride is calculated to ensure that threads evenly cover the entire input array, jumping over elements spaced by the total number of threads in the grid. Each thread iteratively compares elements within its stride, maintaining a temporary maximum (tempMax). Using a stride ensures that all elements get processed when there is a lesser number of threads versus elements. This step reduces the number of elements each thread considers to a single local maximum. The local maximum found by each thread is stored in a shared memory array (cache) within the block. The use of shared memory significantly speeds up the access times compared to global memory. Threads collaboratively perform a reduction operation within the block to find the block's maximum value. This is achieved through a series of steps where threads compare and update shared memory values, effectively halving the number of comparisons needed with each iteration until the maximum value is found. It is important that the threads within the block are synchronized to avoid race conditions.

The final maximum value found by the block is written to a result array at a position corresponding to the block's index. This step requires only the first thread (threadIdx.x == 0) in each block to avoid redundant writes. The kernel as provided computes block-wise maximums. To find the global maximum, a subsequent reduction step is needed to reduce the block maximums to a single overall maximum

4.1 Example

Let's consider an array of integers and how this code would find its maximum value. Assume we have an array A with 8 elements:

A =	3	1	7	8	4	2	6	5

And we configure our kernel launch with 1 block of 4 threads (blockDim.x = 4 and gridDim.x = 1). Each of the 4 threads will start at a different position in A and jump in strides of 4 (the total thread count), meaning each thread checks two elements:

- Thread 0 checks `A[0]` and `A[4]`, finds local max = 4.
- Thread 1 checks `A[1]` and `A[5]`, finds local max = 2.
- Thread 2 checks `A[2]` and `A[6]`, finds local max = 7.
- Thread 3 checks `A[3]` and `A[7]`, finds local max = 8.

The threads store their local maxima in cache and perform reduction. After the first step of reduction, the comparisons are between cache[0] and cache[2], and cache[1] and cache[3]. The new values in cache will be [7, 8, 7, 8]. In the final step, thread 0 compares cache[0] with cache[1], resulting in cache[0] being updated to 8. Thread 0 writes the final maximum value (8) to result[0].

5 Results

The three different methods ares compered based on processing time for a increasing number of elements. Results can be seen in figure 1. The GPU atomic method shows a relatively consistent processing time across different array sizes. This can be attributed to the nature of atomic operations, which, despite their potential for creating bottlenecks, provide a predictable execution pattern. Atomic operations serialize access to shared variables, leading to a processing time that is less affected by the variability in data size. The GPU reduction method shows noticeable peaks. These can result from several factors, such as the inefficient memory access patterns at certain data sizes, the overhead of synchronizing threads during the reduction process, and potentially suboptimal configuration of thread blocks for specific sizes. Interestingly, both methods show a significant dip in processing time at a higher array size. larger data sizes might trigger a more efficient memory access pattern or better utilization of the GPU's computational resources. It's also possible that this specific data size aligns well with the GPU's internal optimizations, such as caching mechanisms, leading to a temporary increase in performance.

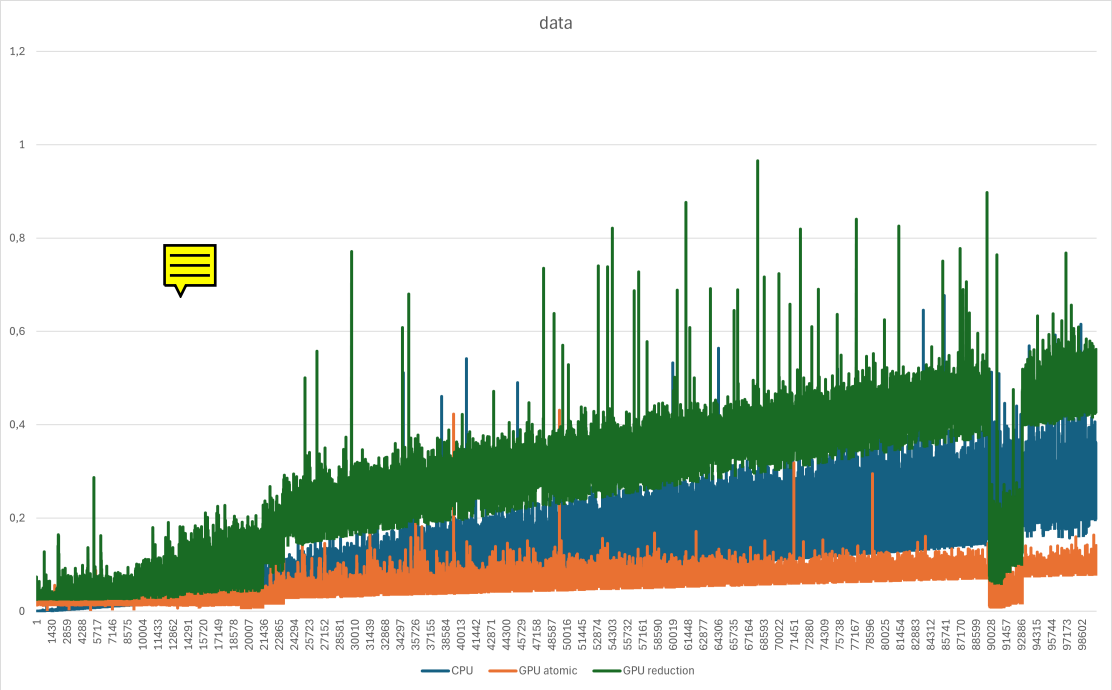


Figure 1: processing time (ms) of kernels for n number of elements